

Information Systems Security

Identification, Authentication and Access
Control

Learning Outcomes

- By the end of this lecture, students should be able to:
 - Explain the difference between **identification, authentication, authorization and accountability**.
 - Apply different authentication mechanisms to appropriate information-system scenarios.
 - Analyse the strengths and weaknesses of passwords, biometrics, tokens, OTPs and MFA.
 - Evaluate authentication systems against common attacks such as brute force, credential stuffing, password spraying and phishing.
 - Apply appropriate access-control principles, including **least privilege, separation of duties, RBAC and ABAC**.

Why Authentication Matters

- Consider a university information system.
 - Thousands of users may access:
 - Student records
 - Examination results
 - Course registration
 - Financial information
 - Research information
 - Staff information
- The system needs to answer four questions:
 - Who are you?
 - Can you prove who you are?
 - What are you allowed to do?
 - What did you do?
- These correspond broadly to:
 - **Identification** → **Authentication** → **Authorization** → **Accountability**

Identification

- **Definition**
- **Identification** is the process by which a user or entity claims an identity.
- Examples:
 - Username
 - Student ID
 - Employee ID
 - Email address
 - Account number
- For example: Username: paul
 - The user is saying: “ I am Paul”
 - However, the system has not yet verified this claim.

Authentication

- **Authentication** is the process of verifying that a claimant is actually associated with the identity being claimed.
- Example:
 - Username: paul
 - Password: xxxxx
- The system checks whether the supplied authenticator corresponds to the account.
- Therefore:
 - **Identification = claiming an identity**
 - **Authentication = proving the identity**

Authorization

- After authentication, the system must determine:
 - What is this user allowed to access?
 - This is **authorization**
- Example:
 - A student may be authorized to:
 - View their own results
 - Register for courses
 - Download their transcript
 - But may not be authorized to:
 - Modify another student's results
 - Create administrator accounts
 - Access the examination database

Accountability

- **Accountability** means being able to associate actions performed within an information system with the responsible user or entity.
- Example:
 - User: student123
 - Time: 10:32
 - Action: Downloaded transcript
 - IP: 192.168.1.25
 - Result: Successful
- This is supported by:
 - Logging
 - Monitoring
 - Audit trails

The Four Concepts

Concept	Main Question	Example
Identification	Who are you claiming to be?	Username
Authentication	Can you prove it?	Password + MFA
Authorization	What can you do?	View own results
Accountability	What did you do?	Audit log

Authentication Factors

- Authentication factors are categories of evidence used to establish identity.
- The commonly recognised factors include:
 1. **Something you know** (Password, PIN, Passphrase)
 2. **Something you have** (Smartphone, OTP generator, Digital Certificate)
 3. **Something you are** (biometric authentication)
 4. **Somewhere you are** (IP Address, Geofencing)
 5. **Something you do** (Typing patterns, Touchscreen behaviour)
- OWASP identifies these five categories, although the first three are the most commonly used in web applications.

Passwords

- Passwords remain one of the most widely used authentication mechanisms.
- A password system has several components:
 - User
 - Authentication Sever
 - Password Verification
 - Access Granted/Denied
- The important security question is: How should the password be stored?

Password Storage

- Passwords should be protected using password hashing mechanisms designed specifically for password storage.
- Passwords should normally be hashed, not reversibly encrypted.
- **Hashing**
 - A one-way transformation: The original password should not be recoverable from the hash.
- Encryption
 - Encryption is reversible when the correct key is available.
- Password Salt
 - A **salt** is a unique random value associated with a password before hashing.
 - Without salts, two users with the same password could produce the same stored hash.
 - This makes precomputed password attacks much more difficult.

Password Hashing

- Hashing (SHA-256) is designed to be **fast**.
- For password storage, attackers may perform huge numbers of guesses.
 - Brute force, Dictionary attack, Password spraying (Try one common password against many accounts), Credential stuffing (Use previously leaked username/password combinations against another service), Phishing
- Password hashing should deliberately be:
 - Slow, Computationally expensive, Preferably memory-intensive
- OWASP specifically recommends password hashing algorithms such as Argon2id rather than fast general-purpose hashes such as SHA-256 for password storage

Countermeasures Against Password Attacks

- Organisations can implement:
 - MFA
 - Password managers
 - Strong password/passphrase policies
 - Breached-password screening
 - Login throttling
 - Rate limiting

Multi-Factor Authentication – MFA

- **MFA** requires authentication using more than one **independent factor category**.
 - Something You Know + Something You Have ☐ MFA
 - Password + Authentication App (U Have) ☐ MFA
 - Password + Fingerprint (U ARE) ☐ MFA
 - Password + Pin is not MFA (Something You Know)

Factor

Examples

Know

Password, PIN

Have

Phone, security key, token

Are

Fingerprint, face

Somewhere

Location/IP

Do

Typing behaviour

Single Sign-On – SSO

- **Single Sign-On** allows a user to authenticate once and access multiple services.
- The user does not need a separate password for every service.
- **Benefits**
 - Improved user experience
 - Centralised authentication
 - Easier account management
 - Centralised security policies
- **Risks**
 - If the central identity provider is compromised: Multiple services may be affected.
- Therefore, identity infrastructure becomes a highly critical asset.

Federated Identity

- Federated identity allows one identity provider to authenticate users for another service.
- Common technologies include:
 - SAML
 - OAuth 2.0
 - OpenID Connect
- OAuth vs Authentication
 - OAuth is primarily an authorization framework, not an authentication protocol.
 - OAuth allows an application to obtain delegated access to resources.
 - For authentication and identity information, **OpenID Connect (OIDC)** is commonly used on top of OAuth 2.0.

Secure Authentication Practices

- A secure authentication system should consider:
 - **Passwords**
 - Do not store plaintext passwords.
 - Use appropriate password hashing.
 - Use unique salts.
 - Allow long passphrases.
 - Block common/compromised passwords.
 - **MFA**
 - Use independent factors.
 - Protect MFA recovery.
 - Protect high-value accounts with MFA.
 - **Login**
 - Use TLS.
 - Rate-limit attempts.
 - Monitor failures.
 - Avoid unnecessarily revealing whether an account exists.
 - **Recovery**
 - Use secure password-reset mechanisms.
 - Use unpredictable, single-use reset tokens.
 - Expire reset tokens.
- OWASP recommends secure password recovery mechanisms and cryptographically secure, single-use, expiring reset tokens.

Access Control

- Once a user is authenticated: What resources can the user access?
- Important models include:
 1. Discretionary Access Control – DAC
 2. Mandatory Access Control – MAC
 3. Role-Based Access Control – RBAC
 4. Attribute-Based Access Control – ABAC

Discretionary Access Control – DAC

- The owner of a resource controls access.
- **Advantage**
 - Flexible.
- **Disadvantage**
 - Users may unintentionally provide excessive access.

Mandatory Access Control – MAC

- Access is determined by centrally defined security policies and classifications.
- MAC is common in environments requiring strict information classification.
- Example: Top Secret, Secret, Confidential, Public
- A user with Secret Clearance may not access Top Secret unless the policy allows it

Role-Based Access Control – RBAC

- Permissions are assigned to **roles**, and users are assigned to roles.
- **Example**
 - **Student**
 - View own results
 - Register courses
 - **Lecturer**
 - View assigned students
 - Enter marks
 - **Administrator**
 - Manage student accounts
 - Generate reports

Advantages of RBAC

- Easier administration
- Supports least privilege
- Easier employee onboarding
- Easier employee transfer
- Easier employee termination
- Supports separation of duties
- Example:
 - Instead of assigning 25 permissions to each employee.
 - The role provides the appropriate permissions.

46. Attribute-Based Access Control – ABAC

- ABAC makes decisions based on attributes.
 - Attributes may include:
 - **User**
 - Role
 - Department
 - Position
 - **Resource**
 - Classification
 - Owner
 - Type
 - **Environment**
 - Location
 - Time
 - Device security status
- Example:
 - IF User.Role = Lecturer AND Resource.Course = User.AssignedCourse AND Time = WorkingHours THEN Allow
 - Else Deny

Principle of Least Privilege

- Users should receive: Only the permissions necessary to perform their legitimate duties.
 - A lecturer who teaches Computer Networks does not necessarily need:
 - Delete Student Account
 - They might only need:
 - View Class list
 - Enter marks